IDENTIDAD ARTIFICIAL

Software Requirements Specification

v1.0 — Abril 2026

Proyecto	Identidad Artificial — Web autónoma sobre IA y tecnología
Versión	1.0
Fecha	Abril 2026
Autor del SRS	Ignacio Cubelas
Stack técnico	Astro + Tailwind CSS + MDX + Cloudflare Pages + GitHub
Agente de contenido	Claude (Anthropic) vía Claude Code
IDE de desarrollo	Google Antigravity
Estado	Borrador — pendiente de validación

1. Introducción

1.1 Propósito del documento

Este documento define los requisitos funcionales, no funcionales y de arquitectura del proyecto Identidad Artificial. Su objetivo es servir como contexto de referencia para los agentes de IA (Claude Code, Google Antigravity) que generarán el código del sitio, y como guía de decisiones técnicas y editoriales durante el desarrollo.

1.2 Alcance del proyecto

Identidad Artificial es un sitio web estático sobre inteligencia artificial, tecnología y tendencias digitales. Su característica diferenciadora es que el contenido es generado de forma autónoma por modelos de lenguaje (Claude), con revisión humana previa a la publicación. El propio sistema de generación es parte del producto: la web documenta cómo funciona el pipeline y expone la arquitectura técnica al lector.

1.3 Definiciones clave

Pipeline	Flujo automatizado: Claude genera post → archivo .mdx → commit → push → despliegue en Cloudflare
Artifact	Post publicado, generado por IA, con metadatos de transparencia (modelo, fecha, prompt base)
Agente	Instancia de Claude Code ejecutando tareas de generación o mantenimiento del sitio
MDX	Markdown extendido con soporte para componentes React/Astro inline
Frontmatter	Metadatos YAML al inicio de cada archivo .mdx (título, fecha, categoría, modelo usado, etc.)

2. Descripción general del sistema

2.1 Perspectiva del producto

El sitio funciona como laboratorio público de IA aplicada al periodismo y la creación de contenido. No es un blog de opinión personal ni un agregador de noticias. Es un experimento replicable donde la arquitectura es tan visible como el contenido.

2.2 Usuarios objetivo

- Desarrolladores e ingenieros de software interesados en sistemas de IA autónomos
- Profesionales de tecnología que consumen contenido sobre tendencias de IA
- Investigadores y estudiantes que quieren ver implementaciones reales de agentes

- Creadores de contenido que evalúan automatización editorial

2.3 Propuesta de valor

La mayoría de webs de contenido IA son indistinguibles entre sí. Identidad Artificial se diferencia en tres ejes:

- Transparencia radical: cada post muestra modelo usado, fecha de generación y prompt base
- Arquitectura visible: una página técnica explica el pipeline completo con código real
- Repositorio público: cualquier lector puede ver, auditar o replicar el sistema

2.4 Restricciones generales

- Sin base de datos. Todo el contenido es estático (archivos .mdx en el repositorio)
- Sin panel de administración. La publicación se hace vía Git
- Sin JavaScript en el cliente por defecto. Astro sirve HTML puro salvo componentes interactivos explícitos
- Sin newsletter en v1.0. Se puede añadir en versiones posteriores
- Las API keys nunca van en el repositorio. Se gestionan como variables de entorno en Cloudflare

3. Arquitectura del sistema

3.1 Stack tecnológico

Framework	Astro (última versión estable)
Estilos	Tailwind CSS v4
Formato de contenido	MDX (.mdx) en src/content/blog/
Despliegue	Cloudflare Pages (build: npm run build, output: dist)
Repositorio	GitHub (público)
Agente de contenido	Claude vía Claude Code
IDE de desarrollo	Google Antigravity
Analytics	Cloudflare Web Analytics (sin cookies, GDPR-friendly)
RSS	@astrojs/rss — automático
Sitemap	@astrojs/sitemap — automático

3.2 Estructura de directorios

```
identidad-artificial/

├── src/

│   ├── content/

│   │   └── blog/          ← Posts en .mdx

│   ├── components/        ← Componentes Astro reutilizables

│   ├── layouts/           ← Layout base y layout de post

│   ├── pages/             ← Rutas: index, sobre, archivo, como-funciona

│   └── styles/            ← CSS global + variables Tailwind

├── public/                ← Assets estáticos (imágenes, favicon)

├── astro.config.mjs

└── package.json
```

3.3 Pipeline de generación de contenido

El flujo de publicación de un post nuevo es el siguiente:

- 1. Se define el tema del post (manualmente o por trigger externo)
- 2. Claude Code recibe el prompt con el tema, el frontmatter template y las instrucciones de estilo
- 3. Claude genera el archivo .mdx completo con frontmatter, cuerpo y metadatos de transparencia
- 4. Revisión humana del contenido antes de publicar (v1.0: obligatoria)
- 5. git add . && git commit -m 'post: [título]' && git push
- 6. Cloudflare Pages detecta el push, ejecuta npm run build, despliega en ~60 segundos

3.4 Frontmatter estándar de cada post

```
---

title: 'Título del post'

description: 'Descripción para SEO y cards'

pubDate: 2026-04-21

category: 'inteligencia-artificial'

tags: ['llm', 'agentes', 'automatizacion']
```

```
heroImage: '/images/post-slug.jpg'

generatedBy: 'claude-sonnet-4-6'

generatedAt: '2026-04-21T10:00:00Z'

promptBase: 'Explica qué son los agentes de IA y...'

humanReviewed: true

---
```

4. Requisitos funcionales

4.1 Páginas del sitio

RF-01 · Home (/)

Página principal. Muestra los últimos posts en orden cronológico inverso. Incluye hero con descripción del proyecto, grid de posts con imagen, título, categoría, fecha y extracto. Paginación si hay más de 12 posts.

RF-02 · Post individual (/[slug]/)

Página de artículo completo. Incluye: título, fecha, categoría, tiempo de lectura estimado, imagen de cabecera, cuerpo en MDX renderizado, y bloque de transparencia al pie con modelo usado, fecha de generación y prompt base.

RF-03 · Sobre el proyecto (/sobre/)

Página estática. Explica qué es Identidad Artificial, quién hay detrás (Ignacio Cubelas) y cuál es el propósito del proyecto. Tono directo, sin marketing vacío.

RF-04 · Cómo funciona (/como-funciona/)

Página técnica. Documenta el pipeline completo: stack, flujo de generación, rol de Claude Code, estructura del repositorio, variables de entorno, y decisiones de diseño. Incluye bloques de código real. Enlaza al repositorio público de GitHub.

RF-05 · Archivo (/archivo/)

Lista paginada de todos los posts, filtrables por categoría y etiqueta. Ordenados por fecha descendente.

RF-06 · RSS (/rss.xml)

Feed RSS completo generado automáticamente con @astrojs/rss. Incluye los últimos 20 posts con título, descripción, fecha y enlace canónico.

4.2 Tabla de requisitos funcionales

ID	Requisito	Prioridad	Estado
RF-01	Página Home con lista de posts recientes y hero	**Alta**	Pendiente
RF-02	Página de post individual con bloque de transparencia	**Alta**	Pendiente
RF-03	Página Sobre el proyecto	**Alta**	Pendiente
RF-04	Página técnica Cómo funciona	**Alta**	Pendiente
RF-05	Página Archivo con filtros por categoría	**Media**	Pendiente
RF-06	RSS automático	**Alta**	Pendiente
RF-07	Sitemap automático	**Alta**	Pendiente
RF-08	Bloque de transparencia en cada post	**Alta**	Pendiente
RF-09	Enlace al repo de GitHub visible en la web	**Media**	Pendiente
RF-10	Tiempo de lectura estimado en cada post	**Baja**	Pendiente

5. Requisitos no funcionales

5.1 Rendimiento

- Lighthouse Performance score ≥ 95 en mobile y desktop
- Largest Contentful Paint (LCP) < 2.0s en conexión 4G simulada
- 0 KB de JavaScript enviado al cliente por defecto (Astro island architecture)
- Imágenes servidas en formato WebP con lazy loading

5.2 SEO

- Metaetiquetas title y description únicas en cada página
- Open Graph y Twitter Card en todos los posts
- URLs canónicas en todas las páginas
- Sitemap XML automático registrado en Google Search Console
- Structured data (JSON-LD) en posts individuales

5.3 Seguridad

- Sin panel de administración expuesto. Superficie de ataque prácticamente nula
- API keys gestionadas exclusivamente como variables de entorno en Cloudflare Pages
- Repositorio público sin ninguna credencial ni secret en el código

- Headers de seguridad configurados en Cloudflare: CSP, X-Frame-Options, HSTS

5.4 Mantenibilidad

- Cada componente Astro tiene una única responsabilidad
- El frontmatter de cada post es la única fuente de verdad de sus metadatos
- El pipeline de generación está documentado en /como-funciona/ y en el README del repo
- Variables de configuración global centralizadas en un archivo src/config.ts

5.5 Accesibilidad

- Contraste de color AA según WCAG 2.1 en todo el texto
- Atributos alt en todas las imágenes
- Navegación funcional sin JavaScript
- Semántica HTML correcta: article, main, nav, header, footer

6. Directrices de diseño

6.1 Principios

- Limpio y técnico. El contenido es el protagonista, no el diseño
- Sin efectos decorativos innecesarios. Nada que no aporte legibilidad
- Tipografía generosa. Interlineado amplio, márgenes cómodos
- Dark mode como opción, no como obligación
- Diferenciado visualmente del WordPress anterior. No copiar el estilo actual

6.2 Paleta de referencia (punto de partida)

Fondo	#FFFFFF (claro) / #0F0F0F (oscuro)
Texto principal	#111111 / #F0F0F0
Texto secundario	#666666 / #999999
Acento	A definir — preferiblemente un color con carácter, no azul genérico
Código inline	Fondo #F4F4F4, fuente monoespaciada

6.3 Tipografía

Titulares	Variable sans-serif (Inter, Geist o equivalente). Peso 700-800

Cuerpo	Misma familia, peso 400. Tamaño 17-18px, interlineado 1.7
Código	JetBrains Mono o equivalente monoespaciada
Carga	Fuentes servidas desde Google Fonts o self-hosted para evitar dependencias externas

7. Fuera de alcance — v1.0

Los siguientes elementos quedan explícitamente excluidos de la primera versión para evitar scope creep:

- Newsletter o sistema de suscripción por email
- Sistema de comentarios
- Área privada o autenticación de usuarios
- Búsqueda interna (se puede añadir con Pagefind en v1.1)
- Internacionalización (i18n)
- E-commerce o monetización
- Pipeline de generación completamente autónomo sin revisión humana (v1.0 requiere aprobación manual)

8. Criterios de aceptación — v1.0

El proyecto se considera completado en v1.0 cuando:

- La web está desplegada en identidadartificial.com vía Cloudflare Pages
- Hay al menos 3 posts publicados con el frontmatter completo incluyendo metadatos de transparencia
- La página /como-funciona/ documenta el pipeline real con código
- El repositorio es público y el README explica cómo replicar el sistema
- RSS y sitemap están activos y validados
- Lighthouse Performance ≥ 90 en mobile
- No hay credenciales expuestas en el repositorio

9. Instrucciones para el agente de desarrollo

Esta sección es específica para Claude Code y Google Antigravity. Léela antes de generar cualquier archivo.

9.1 Orden de trabajo recomendado

- 1. Inicializar proyecto Astro con plantilla blog: npm create astro@latest
- 2. Instalar Tailwind CSS v4 y configurar integración con Astro
- 3. Configurar src/content/config.ts con el schema de frontmatter definido en §3.4
- 4. Crear layout base (BaseLayout.astro) y layout de post (PostLayout.astro)
- 5. Implementar páginas en el orden de prioridad: Home → Post → Sobre → Cómo funciona → Archivo
- 6. Añadir @astrojs/rss y @astrojs/sitemap
- 7. Crear componente TransparencyBlock.astro para el bloque de metadatos de IA en posts
- 8. Añadir 3 posts de ejemplo con frontmatter completo para validar el sistema
- 9. Verificar build limpio con npm run build antes de hacer push

9.2 Reglas que no se negocian

- Nunca hardcodear API keys ni secrets en el código
- Nunca usar JavaScript en el cliente salvo que sea estrictamente necesario para un componente interactivo
- Nunca instalar dependencias que no estén justificadas en este documento
- Nunca modificar la estructura de frontmatter sin actualizar el schema en content/config.ts
- El bloque de transparencia es obligatorio en todos los posts. No es opcional
- Las URLs deben ser limpias y consistentes: /slug/ con barra final

9.3 Cuando tengas dudas

Si algo no está especificado en este documento, la decisión por defecto es: lo más simple que funcione correctamente. No añadas funcionalidad no pedida. No elijas la opción más compleja. Documenta en un comentario por qué tomaste la decisión.

— Fin del documento —